

Logistic Regression: From First Principles

Why Do We Need Logistic Regression?

Linear Regression is used to predict **continuous numerical values** such as:

- House Price
- Salary
- Temperature
- Sales

Now imagine your manager gives you a completely different problem.

|"Build a model that tells whether an email is Spam or Not Spam."

At first, you might think,

|"Why can't I just use Linear Regression?"

This is exactly the question that led to the invention of Logistic Regression.

Let's Try Using Linear Regression

Suppose we define:

- Spam = 1
- Not Spam = 0

Now we train a Linear Regression model. A new email arrives. The model predicts 0.92. That seems reasonable. We can say there is a **92% chance** that the email is spam. Another email arrives. The model predicts 0.08. Again, this makes sense. There is an **8% chance** that the email is spam.

So far, everything looks fine.

The Problem

Now another email arrives. The model predicts 1.80. What does that mean?

Can an email be

180% Spam? No. That doesn't make sense. Now another email arrives. The model predicts -0.35. Can an email be **-35% Spam?** Again, no.

Why Did This Happen?

Nothing is wrong with Linear Regression. It is simply doing what it was designed to do. Remember, Linear Regression predicts **numbers**. It has **no restriction** on the output. It can predict

- -100
- -5
- 0.5
- 10
- 250

This is perfectly fine when predicting:

- House Prices
- Salaries
- Sales

because these values can take almost any number. But classification problems are different.

What is a Classification Problem?

A classification problem is one where the answer belongs to a **category** instead of being any numerical value. Examples include:

Problem	Possible Outputs
Spam Detection	Spam / Not Spam
Disease Detection	Disease / No Disease
Fraud Detection	Fraud / Not Fraud
Loan Approval	Approve / Reject

Notice something. There are only a few possible answers. Not infinitely many numbers.

What Do We Actually Need?

Instead of predicting

-3 or 5.8 or 100, we need something that tells us **How likely** an email is to be spam. For example,

- 95% chance of Spam
- 80% chance of Spam
- 15% chance of Spam
- 2% chance of Spam

These are called **Probabilities**.

What is Probability?

Probability tells us

How likely an event is to happen.

Probability always lies between 0 and 1. or equivalently, 0% to 100%. For example, 0 means impossible. 0.25 means a 25% chance. 0.50 means a 50% chance. 0.90 means a 90% chance. 1 means certain.

Notice something important. Probability can **never** be

- Less than 0
- Greater than 1

This is exactly what we need.

The Solution

We need a model that

- Can never predict below 0.
- Can never predict above 1.
- Always produces a probability.

That model is called

Logistic Regression.

Linear Regression vs Logistic Regression

Linear Regression	Logistic Regression
Predicts continuous numbers	Predicts probabilities
Output can be any value	Output is always between 0 and 1
Used for Regression	Used for Classification

Logistic Regression is a supervised machine learning algorithm used for classification problems. Instead of predicting a continuous numerical value, it predicts the **probability** that an observation belongs to a particular class.

The Sigmoid Function

The problem was simple. Linear Regression can predict

- -5

- 2.8
- 100

These values are perfectly fine when predicting house prices or salaries. But they make no sense when predicting probabilities. A probability can never be

- Less than 0
- Greater than 1

The question is:

How can we force every prediction to stay between 0 and 1?

The answer is the **Sigmoid Function**.

Imagine a Water Filter

Imagine water flowing through a pipe. The amount of water entering the pipe can be anything. A little water. A huge amount of water. Now imagine placing a special filter at the end of the pipe. No matter how much water enters, the filter controls the output. The output can never exceed its maximum limit. The Sigmoid Function works in a similar way. No matter what number enters, the output is always between 0 and 1.

What Does the Sigmoid Function Do?

The Sigmoid Function is simply a converter. Input can be any real number, and output is always a probability between 0 and 1. That's all it does.

The Sigmoid Function converts any real number into a probability between 0 and 1. This is why Logistic Regression uses it.

The Formula

The Sigmoid Function is

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

At first, this formula looks scary. Don't worry. Right now, you do **not** need to memorize it. The important thing is understanding **what it does**, not how it is written.

Suppose the input is

$z = -100$. The output becomes approximately 0.

Suppose

$z = -5$. Output ≈ 0.007 .

Suppose

$z = 0$. Output = 0.5.

Suppose

$z = 5$. Output ≈ 0.993 .

Suppose

$z = 100$. Output ≈ 1 . Notice something amazing. The input changed from -100 all the way to 100, but the output **never** went below 0 or above 1. Exactly what we wanted.

Why is 0.5 So Important?

Look carefully. When $z = 0$, the Sigmoid Function returns 0.5. Think of this as the middle point. If the probability is 0.90, the model is strongly leaning toward Class 1. If the probability is 0.10, the model is strongly leaning toward Class 0. The value 0.5 sits exactly in the middle. That is why we usually use 0.5 as the default decision threshold.

What Happens as z Changes?

Suppose

z becomes larger and larger. For example, 10 20 50 100 The probability gets closer and closer to 1. It never becomes 1.2 or 5. It simply approaches 1.

Now imagine

z becomes more and more negative. -10 -20 -50 -100 The probability gets closer and closer to 0. It never becomes -1 or -10.

The Shape of the Sigmoid Function

If we draw the Sigmoid Function, it looks like a stretched "S".

The sigmoid curve starts near 0, rises smoothly through 0.5, and then flattens near 1. This shape is useful because it turns any real number into a valid probability.

Notice three things.

1. Very negative numbers become close to 0.
2. Very positive numbers become close to 1.
3. The center of the curve is at

0.5.

Where Does z Come From?

So far, we've only talked about z . But where does it come from?

Remember Linear Regression?

It used the equation

$$\hat{y} = \theta_0 + \theta_1 x$$

Logistic Regression starts the same way. It first calculates

$$z = \theta_0 + \theta_1 x$$

Notice something. Nothing has changed yet. The only difference is what happens next. Instead of using z as the final prediction, we pass it into the Sigmoid Function.

The Complete Process

Features go into the linear equation, the model calculates $z = \theta_0 + \theta_1 x$, z passes through the Sigmoid Function, and the output becomes a probability between 0 and 1.

Linear Regression vs Logistic Regression

Linear Regression follows this path: features, equation, prediction. Logistic Regression adds one extra step: features, equation, Sigmoid Function, probability.

The only extra step is **The Sigmoid Function**.

Decision Boundary

For example, 0.15 0.42 0.78 0.95 These numbers tell us **how likely** an observation belongs to Class 1. But there is still one problem.

Imagine your manager asks,

■ "Is this email Spam or Not Spam?"

Would you answer,

■ "There is a 78% chance that it is spam."

Probably not. Your manager wants a simple answer.

■ Spam

or

■ Not Spam

The question becomes:

How do we convert a probability into a final decision?

The answer is called the **Decision Boundary**.

Imagine a School Exam

Suppose the passing marks are 50%. Now imagine four students.

Student	Marks	Result
A	42	Fail
B	49	Fail
C	50	Pass
D	90	Pass

Notice something. The school has drawn an invisible line at 50%. Below it, you fail. At or above it, you pass. That invisible line is called a **Decision Boundary**. Logistic Regression works exactly the same way.

What is a Decision Boundary?

A **Decision Boundary** is the threshold used to convert a probability into a class. The default threshold is 0.5

How Does It Work?

Suppose our model predicts these probabilities.

Probability	Final Prediction
0.10	Class 0
0.25	Class 0
0.49	Class 0
0.50	Class 1
0.80	Class 1
0.99	Class 1

The rule is simple. The rule is simple: if Probability ≥ 0.5 , predict Class 1. If Probability < 0.5 , predict Class 0.

Why 0.5?

Remember the Sigmoid Function. When $z = 0$, Sigmoid gives 0.5. That is exactly the middle of the probability scale. So, 0.5 naturally becomes the default point where the model switches from Class 0 to Class 1.

Does It Always Have to Be 0.5?

No. This is one of the biggest misconceptions. The decision boundary depends on the business problem.

Example 1 – Spam Detection

Suppose the model predicts

0.60 or 60% chance of Spam. Most companies would classify this as Spam. Using 0.5 works well.

Example 2 – Cancer Detection

Now imagine a hospital. The model predicts 35% chance of cancer. Should the doctor ignore the patient?

Probably not. Missing a cancer patient can be life-threatening. The hospital may reduce the threshold. Instead of 0.5, they may choose 0.3. Now, any patient with 30% or more probability will receive additional testing. This increases the chance of detecting cancer early.

Example 3 – Loan Approval

Now imagine a bank. The model predicts 60% chance that a customer will repay the loan. Would the bank approve it?

Maybe not. Losing money is expensive. The bank may increase the threshold. Instead of 0.5, they may require 0.8. Only customers with at least 80% probability will receive approval.

One Model, Different Businesses

Notice something. The model never changed. Only the decision boundary changed. Different businesses choose different thresholds because different mistakes have different costs.

The Complete Prediction Process

The complete prediction process is: features are passed into the equation, the model calculates $z = \theta_0 + \theta_1 x$, the Sigmoid Function converts z into a probability, the Decision Boundary compares that probability with the threshold, and the model predicts the final class.

Binary Cross-Entropy (Log Loss)

A new question arises.

How does the model know whether its prediction is good or bad?

Remember Linear Regression. After making predictions, we calculated the error using **Mean Squared Error (MSE)**. Can we use the same Cost Function here?

The answer is **No**. Let's understand why.

Suppose an email is actually Spam. So, Actual Value

$$y = 1$$

Now imagine two models.

Model A

Predicts

0.90

Model B

Predicts

0.20

Which model is better?

Obviously, Model A. It predicted a 90% chance of Spam, which is much closer to the correct answer. So our Cost Function should

- Give a **small penalty** to Model A.
- Give a **large penalty** to Model B.

Can We Use MSE?

Remember the formula.

$$MSE = (y - \hat{y})^2$$

Let's calculate it.

Model A

Actual = 1 Prediction = 0.90 Error

$$1 - 0.90 = 0.10$$

Square it.

$$0.10^2 = 0.01$$

Very small error. Looks good.

Model B

Actual = 1 Prediction = 0.20 Error

$$1 - 0.20 = 0.80$$

Square it.

$$0.80^2 = 0.64$$

Much larger error. Still looks reasonable. So where is the problem?

The Real Problem

The problem is **not the value of MSE**. The problem is how MSE behaves together with the **Sigmoid Function**. Remember, our prediction is not calculated directly. The model first computes

$$z = \theta_0 + \theta_1 x$$

Then, Sigmoid converts z into a probability. When we combine

- Sigmoid
- MSE

Gradient Descent does **not** learn as efficiently. Learning becomes slower and optimization becomes harder. So researchers looked for a Cost Function designed specifically for probabilities. That Cost Function is **Binary Cross-Entropy**, also called **Log Loss**.

What Do We Really Want?

Forget formulas for a moment. Think logically. Suppose the correct answer is Spam. Now imagine four predictions. Prediction

0.99

Excellent. Almost no penalty.

Prediction

0.80

Still good. Small penalty.

Prediction

0.55

Not very confident. Medium penalty.

Prediction

0.01

Very bad. The model was almost certain the email was **Not Spam**, but it was actually Spam. This should receive a **huge penalty**. That is exactly what Binary Cross-Entropy does.

Another Example

Suppose the actual answer is

0

(Not Spam) Now compare these predictions. Prediction

0.02

Excellent. Very small penalty.

Prediction

0.40

Not great. Moderate penalty.

Prediction

0.98

Terrible. The model was extremely confident, and completely wrong. This deserves a huge penalty. Again, Binary Cross-Entropy behaves exactly like this.

The Formula

The Binary Cross-Entropy Loss is

$$Loss = - [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Don't panic. This formula looks complicated, but right now, you do **not** need to memorize it. The important thing is understanding **what it is trying to achieve**. Later, when we study logarithms, this formula will become much easier to understand.

Why Is It Called Cross-Entropy?

Think of it like this. The model says,

"I believe this email has a 95% chance of being Spam."

Reality says,

"Let's see if you're correct."

Cross-Entropy simply measures **how different the model's predicted probability is from reality**. The closer they match, the smaller the loss. The farther apart they are, the larger the loss.

Linear Regression vs Logistic Regression

Linear Regression	Logistic Regression
Predicts numbers	Predicts probabilities
Uses MSE	Uses Binary Cross-Entropy
Penalizes numerical errors	Penalizes incorrect probabilities

Binary Cross-Entropy (Log Loss) is the Cost Function used in Logistic Regression. It measures how well the predicted probabilities match the actual classes.

Gradient Descent in Logistic Regression

A new question arises.

Once the model knows it made a mistake, how does it improve itself?

The answer is

Gradient Descent.

Let's Remember Linear Regression

In Linear Regression, the model guesses θ values, makes predictions, calculates error, calculates cost, finds the derivative, updates θ , and repeats. Logistic Regression follows the same journey.

Logistic Regression Learning Process

Logistic Regression starts by guessing θ values. It calculates $z = \theta_0 + \theta_1 x$, applies the Sigmoid Function, predicts a probability, calculates Binary Cross-Entropy Loss, finds the derivative, updates θ , and repeats. The only major difference from Linear Regression is the Cost Function: instead of MSE, Logistic Regression uses Binary Cross-Entropy.

Imagine Learning Basketball

Suppose you're learning basketball. You take a shot. You miss. Do you quit?

No. You adjust your aim. Then you shoot again. Still miss. Adjust again. Eventually, your shots start going in. Machine Learning behaves exactly the same way. Every prediction teaches the model how to improve.

What Does Gradient Descent Actually Do?

Think back to Linear Regression. The Cost Function told us **How wrong the model is**. The Derivative told us **Which direction reduces the Cost**. Gradient Descent used that information to update θ . Exactly the same thing happens here.

The Update Rule

The update equation is

$$\theta = \theta - \alpha \frac{\partial J}{\partial \theta}$$

Don't panic. We've already learned this equation. Nothing has changed. Let's understand it again.

θ (Theta)

The parameter we want to improve. Examples θ_0 , θ_1 , θ_2 , and so on.

α (Learning Rate)

Controls the size of each step. Small α means slow learning. Large α may overshoot the minimum.

Derivative

The derivative tells us

which direction decreases the Loss. A positive derivative means move left. A negative derivative means move right. Exactly the same idea as Linear Regression.

What Changed?

Only one thing. In Linear Regression, the derivative came from MSE. In Logistic Regression, the derivative comes from Binary Cross-Entropy. The learning process itself did not change.

One Complete Learning Cycle

Let's imagine one iteration.

Step 1

Random θ values make the model predict 0.35.

Step 2

Actual answer

1 (Spam)

Step 3

Binary Cross-Entropy says "This prediction is not good." Loss is high.

Step 4

Derivative calculates

which direction decreases the Loss.

Step 5

Gradient Descent updates θ .

Step 6

Model predicts again. Now the prediction is 0.48. Better.

Repeat. The prediction becomes 0.67. Better. Repeat. The prediction becomes 0.91. Excellent. The model has learned.

Why Do We Repeat?

One update is rarely enough. Imagine climbing down a mountain in heavy fog. You cannot see the bottom. You take one step, look around, take another step, look again, until you finally reach the bottom. Gradient Descent works exactly like that. It keeps taking small steps toward the minimum Loss.

Does Logistic Regression Ever Stop Learning?

Yes. Training usually stops when

- The Loss stops decreasing significantly.
- The maximum number of iterations is reached.
- The model has converged to a good solution.

At this point, the θ values are considered learned.

Linear Regression vs Logistic Regression

Linear Regression	Logistic Regression
Predict numbers	Predict probabilities
Cost Function = MSE	Cost Function = Binary Cross-Entropy
Uses Gradient Descent	Uses Gradient Descent
Updates θ repeatedly	Updates θ repeatedly

Notice something interesting. The learning algorithm is almost identical. Only the Cost Function changes.

Gradient Descent is an optimization algorithm that repeatedly updates the model parameters (θ) to reduce the Binary Cross-Entropy Loss.

Confusion Matrix

Now the model is trained. The next question is:

How do we know whether the model is actually good?

Suppose we tested our model on 100 emails. The model made predictions for all 100. Some predictions were correct. Some were wrong. Before calculating Accuracy, Precision, or Recall, we first need to organize these predictions. That is the purpose of the **Confusion Matrix**. Everything in classification starts here.

Imagine a Hospital

Suppose a hospital builds an AI model to detect cancer. Every patient belongs to one of two groups. Positive means the patient has cancer. Negative means the patient does not have cancer. The AI also predicts Positive or Negative. Now let's compare the prediction with reality.

There Are Only Four Possibilities

Every prediction must fall into one of four cases. Nothing more. Nothing less.

Case 1 – True Positive (TP)

The patient actually has cancer. The model predicts Cancer. The model is correct. This is called **True Positive**.

A **True Positive** occurs when the actual class is Positive and the model also predicts Positive.

Case 2 – True Negative (TN)

The patient is healthy. The model predicts Healthy. Again, the model is correct. This is called **True Negative**.

A **True Negative** occurs when the actual class is Negative and the model also predicts Negative.

Case 3 – False Positive (FP)

The patient is healthy. The model predicts Cancer. The model made a mistake. This is called **False Positive**. It is also called a **False Alarm**.

A **False Positive** occurs when the actual class is Negative but the model predicts Positive.

Case 4 – False Negative (FN)

The patient actually has cancer. The model predicts Healthy. This is also wrong. But notice something. This mistake is much more dangerous. The patient may not receive treatment. This is called **False Negative**.

A **False Negative** occurs when the actual class is Positive but the model predicts Negative.

The Complete Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

This table is called the **Confusion Matrix**. Every classification metric is calculated from these four numbers.

Example

Suppose we tested

100 patients. The results are $TP = 45$, $TN = 40$, $FP = 10$, $FN = 5$. That means 45 patients had cancer, and the model correctly detected them. 40 healthy patients were correctly identified. 10 healthy patients were wrongly told they had cancer. 5 cancer patients were missed. These four numbers tell the complete story of the model.

Why Is It Called "Confusion" Matrix?

Imagine looking at only the final predictions. You know some were correct and some were wrong. But you don't know **what kind of mistakes** the model made. The Confusion Matrix removes that confusion. It separates every prediction into one of four categories.

Linear Regression vs Logistic Regression

In Linear Regression, we compare the actual value and predicted value, then calculate MAE, MSE, RMSE, and R^2 .

In Logistic Regression, we first build the Confusion Matrix, then calculate Accuracy, Precision, Recall, F1-Score, ROC, and AUC.

A **Confusion Matrix** is a table that compares the model's predictions with the actual classes. It summarizes every prediction into one of four categories:

- True Positive (TP)
- True Negative (TN)
- False Positive (FP)
- False Negative (FN)

Accuracy

Now we want to answer a simple question.

Overall, how good is my model?

The simplest way to answer this question is by using **Accuracy**.

Imagine a Teacher Checking Exams

Suppose a teacher checks

100 exam papers. After checking, the teacher discovers 95 answers are correct. 5 answers are incorrect. Now ask yourself. How well did the student perform?

Most people would say

"The student got 95 out of 100 correct." or "The student scored 95%." That is exactly what Accuracy measures.

What is Accuracy?

Accuracy measures the percentage of predictions that the model got correct. It does not care whether the prediction was Positive or Negative. It simply asks

How many predictions were correct?

The Formula

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Let's understand it.

Numerator

$$TP + TN$$

These are all the **correct predictions**.

- TP = Correct Positives
- TN = Correct Negatives

So, the numerator means total correct predictions.

Denominator

$$TP + TN + FP + FN$$

This is simply

every prediction the model made. correct + incorrect = Total Predictions.

Example 1

Suppose we tested

100 patients. The Confusion Matrix is TP = 40, TN = 50, FP = 5, FN = 5 Let's calculate Accuracy. Correct Predictions = TP + TN = 40 + 50 = 90 Total Predictions = 100 Therefore,

$$Accuracy = \frac{90}{100} = 0.90$$

Accuracy = 90%. This means the model correctly predicted 90 out of 100 patients.

Sounds Great?

Imagine your friend tells you, "My model has 99% Accuracy." Would you immediately believe it's an amazing model?

Probably. But let's look at another example.

Example 2 – When Accuracy Lies

Imagine a hospital. There are 1000 patients. Only 10 actually have cancer. The remaining 990 are healthy. Now imagine a terrible AI model. It predicts **Healthy** for every single patient. Let's calculate the Accuracy. Correct Predictions = 990. Wrong Predictions = 10. Accuracy = $990 \div 1000 = 99\%$. Wow.

99% Accuracy. That sounds incredible. But think carefully. Did the model detect even one cancer patient?

No. It missed all 10 patients. This model is almost useless.

Why Did Accuracy Fail?

Because the data was

imbalanced. There were 990 healthy patients and only 10 cancer patients. The model took the easy path. It simply predicted Healthy for everyone. Accuracy looked excellent, but the model completely failed its real job.

When Should We Use Accuracy?

Accuracy works well when

the classes are **balanced**. For example, suppose we have 500 spam emails and 500 normal emails. Now, Accuracy becomes much more meaningful.

When Should We Avoid Accuracy?

Avoid relying only on Accuracy when one class is much larger than the other. Examples include

- Cancer Detection
- Fraud Detection
- Credit Card Fraud
- Network Intrusion Detection

These datasets are often highly imbalanced.

Linear Regression vs Logistic Regression

Linear Regression asks

"How close are my predictions?"

Logistic Regression asks

"How many predictions were correct?"

Accuracy is the proportion of correct predictions made by the model out of all predictions.

Precision

Imagine an Email System

Suppose your company builds an AI model to detect spam emails. One day, the model predicts 100 emails are Spam. Now imagine you manually check those emails. You discover 90 were actually Spam. But 10 were normal emails. The model incorrectly marked those 10 emails as Spam. Now ask yourself.

When my model said "Spam", how often was it actually correct?

That is exactly what **Precision** measures.

What is Precision?

Precision measures how many predicted positive cases were actually positive. Notice something. Precision does **not** look at all emails. It only looks at the emails that the model predicted as Positive.

The Formula

$$Precision = \frac{TP}{TP + FP}$$

Let's understand it.

Numerator

$$TP$$

These are

the Positive predictions that were actually correct.

Denominator

$$TP + FP$$

These are

all the Positive predictions made by the model. Some were correct. (TP) Some were wrong. (FP)

Example

Suppose our Confusion Matrix is TP = 90, FP = 10, TN = 80, FN = 20 Let's calculate Precision. Correct Positive Predictions = 90 Total Positive Predictions = 90 + 10 = 100 Therefore,

$$\text{Precision} = \frac{90}{100} = 0.90$$

Precision = 90%. This means whenever the model predicts Positive, it is correct 90% of the time.

Another Example

Suppose another model predicts 100 patients have cancer. After testing, we discover only 50 actually have cancer. The other 50 are healthy. Precision becomes

$$\frac{50}{100} = 50\%$$

That means

Half of the patients told "You may have cancer" were actually healthy. Not a very good model.

Why Is Precision Important?

Imagine Gmail. Your model predicts Spam. But the email is actually a job offer. Or an important bank message. Or a university admission letter. The user may never see it. That is a serious problem. In spam detection, False Positives are expensive. So, Precision becomes very important.

Another Real-World Example

Imagine a facial recognition system at an airport. The AI says, "This person is a criminal." Security immediately stops that person. Later, they discover the person is innocent. That is a False Positive. Too many False Positives cause unnecessary trouble. Again, we want high Precision.

What Does High Precision Mean?

High Precision means

When the model predicts

Positive, you can trust it. There are very few False Positives.

What Does Low Precision Mean?

Low Precision means

The model frequently predicts Positive when it should not. There are many False Positives.

Accuracy vs Precision

Accuracy asks

| Out of all predictions, how many were correct?

Precision asks

| Out of everything I predicted as Positive, how many were actually Positive?

Notice the difference. Accuracy looks at **every prediction**. Precision looks at **only Positive predictions**.

Precision is the proportion of correctly predicted Positive cases out of all cases predicted as Positive.

Recall (True Positive Rate)

Now let's look at another important question. Imagine there are 100 patients who actually have cancer. Your AI model correctly detects 90 patients.

But it completely misses

10 patients. Now ask yourself.

| Out of all the patients who actually had cancer, how many did my model find?

That is exactly what **Recall** measures.

What is Recall?

Recall measures how many actual Positive cases were correctly identified by the model. Unlike Precision, Recall does **not** care about False Positives. It only cares about **missing actual Positive cases**.

The Formula

$$Recall = \frac{TP}{TP + FN}$$

Recall is also called

True Positive Rate (TPR). Both names mean exactly the same thing.

Let's Understand the Formula

Numerator

$$TP$$

These are

the Positive cases the model successfully found.

Denominator

$$TP + FN$$

These are

all the actual Positive cases. Some were detected correctly. (TP) Some were missed. (FN)

Example

Suppose

100 patients actually have cancer. The model detects 90. It misses 10. So, TP = 90, FN = 10 Recall becomes

$$Recall = \frac{90}{90 + 10} = \frac{90}{100} = 0.90$$

Recall = 90%. This means the model found 90% of all cancer patients.

Another Example

Suppose

100 spam emails arrived today. Your model detects 70. It misses 30. Then Recall becomes

$$Recall = \frac{70}{100} = 70\%$$

This means your model caught only 70% of the spam emails.

Why Is Recall Important?

Imagine a hospital. A patient actually has cancer. The model predicts Healthy. The patient goes home. Treatment is delayed. This is called a **False Negative**. In healthcare, False Negatives can be life-threatening. That is why hospitals often care much more about **Recall** than Precision.

Another Real-World Example

Imagine airport security. A dangerous person walks through security. The AI says "Everything is fine." That person is missed. Again, this is a False Negative. Security systems usually aim for very high Recall, even if that means stopping a few extra innocent people.

High Recall

High Recall means

the model misses very few actual Positive cases. There are very few False Negatives.

Low Recall

Low Recall means

the model misses many actual Positive cases. There are many False Negatives.

Precision vs Recall

This is one of the most important interview questions. Imagine a fishing net.

Precision asks

| Out of everything I caught, how many were actually fish?

If you catch

90 fish and 10 old shoes, your Precision is high.

Recall asks

| Out of all the fish in the lake, how many did I catch?

Suppose there were

100 fish, but you caught only 60. Your Recall is 60%. Notice the difference. Precision looks at **your catch**. Recall looks at **everything that existed**.

Accuracy vs Precision vs Recall

Accuracy asks

| How many predictions were correct?

Precision asks

When I predict Positive, how often am I correct?

Recall asks

Out of all actual Positives, how many did I find?

Each metric answers a different question.

Recall (True Positive Rate) is the proportion of actual Positive cases that were correctly identified by the model.

False Positive Rate (FPR)

Now let's look at the opposite side. Instead of focusing on the Positive cases, let's focus on the **Negative cases**. The question becomes

Out of all the actual Negative cases, how many did my model wrongly classify as Positive?

This is called the

False Positive Rate (FPR).

What is False Positive Rate?

False Positive Rate measures how many actual Negative cases were incorrectly predicted as **Positive**. Remember, these are **False Alarms**.

The Formula

$$FPR = \frac{FP}{FP + TN}$$

Let's understand it.

Numerator

$$FP$$

These are

the Negative cases the model wrongly predicted as Positive. These are our False Alarms.

Denominator

$$FP + TN$$

These are

all the actual Negative cases. Some were predicted correctly. (TN) Some were wrongly predicted as Positive. (FP)

Example

Suppose

100 healthy patients visit a hospital. The AI predicts 20 patients have cancer. But they are actually healthy. That means FP = 20. The remaining 80 are correctly identified as healthy. TN = 80. Now calculate FPR.

$$FPR = \frac{20}{20 + 80} = \frac{20}{100} = 0.20$$

FPR = 20%. This means the model falsely alarmed 20% of healthy patients.

Another Example

Imagine an email system. There are 500 normal emails. The model incorrectly marks 50 as Spam. FP = 50. The remaining 450 stay in the inbox. TN = 450. FPR becomes

$$FPR = \frac{50}{500} = 10\%$$

This means

10% of genuine emails were wrongly classified as Spam.

Why Is FPR Important?

Imagine airport security. A security scanner stops 100 innocent passengers for additional checking. Only 2 are actually dangerous. The scanner created many False Positives. Everyone waits in longer lines. Time is wasted. People become frustrated. A high FPR can make a system very inconvenient.

High FPR

High FPR means

the model creates many False Alarms. Many Negative cases are wrongly classified as Positive.

Low FPR

Low FPR means

very few innocent cases are wrongly classified as Positive. This is what we usually want.

Recall vs False Positive Rate

These two metrics are closely related. Recall looks at the Positive class. FPR looks at the Negative class.

Recall asks

Out of all actual Positives, how many did I correctly find?

Formula

$$\text{Recall} = \frac{TP}{TP + FN}$$

FPR asks

Out of all actual Negatives, how many did I wrongly classify as Positive?

Formula

$$\text{FPR} = \frac{FP}{FP + TN}$$

Notice something. Recall rewards finding Positive cases. FPR penalizes wrongly accusing Negative cases.

Why Are Recall and FPR Used Together?

Imagine lowering the Decision Boundary. Instead of 0.5, you choose 0.2. What happens?

The model predicts

Positive

much more often. Good news. Recall usually increases. You find more cancer patients. But FPR also increases. Now many healthy people are wrongly told they may have cancer. This is a trade-off. Increasing Recall often increases FPR. Reducing FPR often decreases Recall. Finding the right balance is very important.

False Positive Rate is the proportion of actual Negative cases that were incorrectly predicted as Positive.

F1-Score

Each metric tells us something different about our model. Now imagine we build two models.

Model A

Precision = 95%, Recall = 60%

Model B

Precision = 75%, Recall = 90%. Now the question is:

Which model is better?

The answer is not obvious. Model A has better Precision. Model B has better Recall. So how do we compare them?

The answer is

F1-Score.

Why Do We Need F1-Score?

Imagine two students.

Student A

Math = 100, Science = 20

Student B

Math = 80, Science = 80. Who is the better overall student?

Most people would choose

Student B. Why?

Because Student B is balanced. F1-Score works exactly the same way. It rewards models that perform well in **both Precision and Recall**.

What is F1-Score?

F1-Score is a metric that combines Precision and Recall into a single value. Instead of looking at two numbers, we get one overall score.

The Formula

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Don't worry about memorizing it. The important thing is **what it is trying to do**.

Imagine your model has

Precision = 100%, Recall = 20%. Is this a great model?

Not really. Yes, whenever it predicts Positive, it is always correct. But it only finds 20% of the actual Positive cases. It misses 80%. That is not acceptable.

Now imagine another model. Precision = 20%, Recall = 100%. Again, not a great model. It finds everyone, but it wrongly labels many Negative cases as Positive.

F1-Score says

"I don't care if one metric is excellent and the other is terrible."

"I want both Precision and Recall to be good."

Example 1

Suppose

Precision = 90%, Recall = 90%. The model is balanced. The F1-Score will also be about 90%. Excellent model.

Example 2

Suppose

Precision = 95%, Recall = 40%. Although Precision is very high, Recall is poor. The F1-Score will drop significantly. Why?

Because the model is unbalanced.

Example 3

Suppose

Precision = 60%, Recall = 60%. Neither metric is amazing, but both are similar. The F1-Score will also be close to 60%.

Why Doesn't F1 Just Take the Average?

This is an excellent question. Suppose Precision = 100%, Recall = 0%. The average would be 50%. That sounds reasonable. But think carefully. The model either finds nobody or never predicts correctly. This model is terrible. F1-Score gives it a value close to 0, not 50%. That is why F1 is much better than simply averaging Precision and Recall. It **penalizes imbalance**.

When Should We Use F1-Score?

Use F1-Score when both Precision and Recall are important. Examples

- Fraud Detection
- Disease Detection
- Spam Detection
- Credit Card Fraud
- Fake News Detection

These problems need a balance.

Precision vs Recall vs F1

Precision asks

|"When I predict Positive, how often am I correct?"

Recall asks

|"Out of all actual Positives, how many did I find?"

F1 asks

|"How well do I balance both Precision and Recall?"

Linear Regression vs Logistic Regression

Linear Regression evaluates models using

- MAE
- MSE
- RMSE
- R^2

Logistic Regression evaluates models using

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC

F1-Score is the harmonic mean of Precision and Recall. Its purpose is to measure how well a model balances both metrics.

ROC Curve and AUC

These are excellent evaluation metrics. But they all have one thing in common. They depend on the **Decision Boundary**. For example, suppose we use 0.5 as the Decision Boundary. Our model gives Precision = 90%, Recall = 80%

Now suppose we change the Decision Boundary to 0.7. The model may now produce Precision = 96%, Recall = 65%. Nothing changed in the model. Only the Decision Boundary changed. This raises an important question.

Can we evaluate the model without choosing a Decision Boundary?

The answer is

Yes. That is exactly why the **ROC Curve** was created.

Why Do We Need ROC?

Imagine two AI models. Both classify emails as Spam or Not Spam. Model A works well no matter where you place the Decision Boundary. Model B works well only when the threshold is exactly 0.5. Which model is better?

Obviously, Model A. ROC helps us compare models without fixing a single threshold.

What Does ROC Stand For?

ROC stands for

Receiver Operating Characteristic. Don't worry about the name. The name comes from radar systems during World War II. Today, we only care about what it does.

What Does ROC Measure?

ROC shows

how the model behaves at **every possible Decision Boundary**. Instead of testing only 0.5, ROC checks thresholds such as 0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, and 0.1. Each threshold produces a different Confusion Matrix. Each Confusion Matrix produces a different TPR and FPR. Every pair (TPR, FPR) becomes one point on the ROC Curve.

The Two Axes

The ROC Curve uses only two numbers.

Y-Axis

True Positive Rate

(TPR) This is exactly the same as Recall. Formula

$$TPR = \frac{TP}{TP + FN}$$

Question it answers

Out of all actual Positives, how many did I correctly find?

X-Axis

False Positive Rate

(FPR) Formula

$$FPR = \frac{FP}{FP + TN}$$

Question it answers

Out of all actual Negatives, how many did I wrongly classify as Positive?

One Model Gives Many ROC Points

This is one of the most misunderstood concepts. Suppose our model predicts these probabilities.

Customer	Actual	Probability
A	1	0.95
B	1	0.80
C	1	0.60
D	0	0.55
E	0	0.30
F	0	0.10

Notice something. The model never changes. Only the Decision Boundary changes.

Threshold = 0.90

Only Customer A is predicted Positive. Calculate TPR and FPR. This gives one point.

Threshold = 0.70

Now A and B become Positive. Calculate TPR and FPR again. This gives another point.

Threshold = 0.50

Now

A

B

C

and D become Positive. Again, calculate TPR and FPR. This gives another point.

Repeat this process

for every possible threshold. Joining all those points creates the ROC Curve.

The 45° Line

Every ROC graph contains a diagonal line.

In an ROC curve, the vertical axis represents TPR and the horizontal axis represents FPR. A stronger model bends closer to the top-left area, while a weak model stays closer to the diagonal line.

This diagonal line represents **Random Guessing**. Why?

Because

a random model increases TPR and FPR at almost the same rate. So, $TPR \approx FPR$. A model on this line is no better than flipping a coin.

Above the Line

If the ROC Curve is above the diagonal, the model is learning useful patterns. This is what we want.

Below the Line

If the ROC Curve is below the diagonal, the model is performing worse than random guessing. In many cases, simply reversing its predictions would improve the results.

What is AUC?

Looking at the entire ROC Curve is useful. But sometimes, we want one number to summarize everything. That number is called **AUC** which stands for **Area Under the Curve**.

AUC is the total area under the ROC Curve. It summarizes the model's ability to separate Positive and Negative classes.

Understanding AUC

AUC = 1

Perfect classifier. Every Positive example receives a higher probability than every Negative example.

AUC = 0.5

Random guessing. The model cannot distinguish between the two classes.

AUC < 0.5

Worse than random. Something is likely wrong with the model.

The Deep Meaning of AUC

Imagine you randomly choose one Positive sample and one Negative sample. Now compare their predicted probabilities. If the model gives the Positive sample a higher probability, it wins. Do this thousands of times. AUC is simply the probability that the Positive sample receives a higher score than the Negative sample. This is why AUC measures how well the model **ranks** the two classes.

Accuracy vs ROC-AUC

Accuracy asks

How many predictions were correct?

ROC-AUC asks

How well can my model separate the two classes regardless of the Decision Boundary?

Notice the difference. ROC-AUC does not depend on choosing 0.5 or 0.7 or 0.3. It evaluates the model itself.